

Pesquisa sobre Prepared Statements em PHP com MySQL:

- **O que são Prepared Statements?**

Um Prepared Statement é um recurso de sistemas de gerenciamento de banco de dados usado para executar comandos SQL de forma segura e rápida. Ele funciona como um modelo pré-compilado onde a estrutura do código SQL é enviada e analisada pelo banco de dados separadamente dos parâmetros que serão inseridos depois.

- **Como funciona o fluxo:**

1. Preparação

O código envia para o banco de dados apenas a estrutura do comando SQL, sem os dados finais. Para indicar onde as informações serão inseridas depois, utilizam-se espaços reservados (placeholders), que podem ser pontos de interrogação ? ou marcadores com nome, como :nome.

2. Análise

Assim que o banco de dados recebe esse modelo, ele faz uma leitura completa da estrutura. O sistema confere a sintaxe do código, valida as tabelas e cria um plano para buscar as informações. O ponto principal é que o banco de dados trava a lógica dessa estrutura, garantindo que nenhum dado enviado depois consiga alterar o comando original.

3. Execução

Com o modelo validado e salvo na memória do banco, são enviados apenas os dados reais para preencher os espaços em branco. O banco encaixa os valores nas lacunas correspondentes e roda a consulta imediatamente. Se for necessário repetir a mesma busca com novos dados, não é preciso reenviar a estrutura SQL; o sistema pula as etapas anteriores e envia apenas as novas informações.

- **SQL Injection**

- O que é?

Injeção de SQL é uma das vulnerabilidades mais conhecidas e perigosas na segurança de sistemas web. Esse ataque acontece quando um usuário mal-intencionado consegue inserir comandos SQL próprios dentro dos campos de entrada de um site (como formulários de login ou barras de pesquisa), fazendo com que o banco de dados execute ordens que não deveria.

- Como pode acontecer?

O problema ocorre quando o sistema pega o texto digitado pelo usuário e o mistura diretamente com o código que conversa com o banco de dados. Se o sistema não verificar ou limpar o que foi digitado, o banco de dados não conseguirá diferenciar onde termina a ordem do programador e onde começam os dados do usuário, acabando por ler e executar tudo junto como se fosse uma instrução única.

- Por que é perigoso colocar dados do usuário diretamente no SQL?

Porque permite que o usuário altere a lógica interna do sistema. Por exemplo, se o código for escrito apenas juntando textos e um invasor digitar ' OR '1'='1 no campo de senha, a consulta final enviada ao banco de dados aceitará essa condição como sempre verdadeira. Isso possibilita que criminosos ignorem telas de login, acessem dados confidenciais de outros usuários ou até apaguem tabelas inteiras do banco de dados.

- Como o Prepared Statement ajuda a evitar esse problema?

Ele evita esse problema porque separa de forma rígida a estrutura do comando dos dados enviados. Como o banco de dados já analisou e travou o modelo da consulta antes de

receber os valores, qualquer texto que o usuário digite será tratado puramente como informação literal (um texto comum) e nunca como parte de uma ordem. Mesmo que o invasor tente enviar comandos maliciosos, o banco de dados tentará apenas buscar aquele texto exato, neutralizando o ataque por completo.

- **Fontes da pesquisa:**

[Wikipedia](#)

[W3schools](#)

[Alura](#)

[PHP](#)

Análise do projeto:

O projeto possui as operações de cadastro, consulta, atualização e exclusão de livros.

- **Cadastrar**

Código original:

```
<?php

include "../infra/conexao.php";

$titulo = $_POST["titulo"];
$autor = $_POST["autor"];
$ano = $_POST["ano"];

$sql = "INSERT INTO livros (titulo,autor,ano) VALUES
('$titulo','$autor','$ano')";

mysqli_query($conexao, $sql);

header("Location: ../index.php");
?>
```

Versão modificada:

```
<?php

include "../infra/conexao.php";

$titulo = $_POST["titulo"];
$autor = $_POST["autor"];
$ano = $_POST["ano"];

// query com ? no lugar dos valores
$sql = "INSERT INTO livros (titulo,autor,ano) VALUES (?, ?, ?)";
```

```

$stmt = $conexao->prepare($sql); // prepara a consulta no banco
$stmt->bind_param("ssi", $titulo, $autor, $ano); // liga cada variavel
a um ?
$stmt->execute(); // os dados são enviados

header("Location: ../index.php");
?>

```

Explicação: No código original, os dados que a pessoa digita no formulário (título, autor e ano) são colocados direto dentro do texto do comando SQL junto com aspas simples. O problema disso é que o PHP monta a string toda e manda para o banco como um texto único, e o banco não tem como saber o que é o comando e o que é o dado digitado pelo usuário. Se alguém digitar aspas ou outros caracteres de SQL num desses campos, dá para alterar o comando inteiro acidentalmente. Na versão modificada isso muda porque em vez de colocar os valores direto na string, uso uma interrogação (?) no lugar de cada valor. Isso separa a forma do comando dos dados que vão entrar nele. Para isso, usei três funções:

- prepare(): manda para o banco só a estrutura da consulta, com os ? no lugar dos valores, e o banco já confere se essa estrutura está correta antes de receber qualquer dado.
- bind_param(): liga cada variável (\$titulo, \$autor, \$ano) a um dos ?, na ordem certa, e também avisa o tipo de cada uma (uso "ssi", onde s é texto e i é número inteiro).
- execute(): é quando os dados realmente são enviados pro banco e a consulta roda.

O nome da variável \$stmt vem de "statement" e é o nome que a documentação oficial do PHP usa para essa variável, porque ela guarda a consulta já preparada.

A principal diferença entre as duas formas é que no código original, o usuário e o comando ficam misturados no mesmo texto antes de chegar no banco, então o banco não consegue diferenciar um do outro. Já usando parâmetros, o banco recebe primeiro só o comando travado, e só depois recebe os dados separadamente, e esses dados são sempre tratados como informação pura, nunca como parte do comando. É por isso que mesmo que alguém tente digitar um código malicioso num campo do formulário, ele não teria efeito nenhum, pois o banco só vai procurar por aquele texto exato, e não vai executar ele como se fosse um comando.

● Excluir

Código original:

```

<?php
include "../infra/conexao.php";
$id = $_GET["id"];
$sql = "DELETE FROM livros WHERE id=$id";
mysqli_query($conexao,$sql);
header("Location: ../index.php");
?>

```

Versão modificada:

```

<?php
include "../infra/conexao.php";

```

```

$id = $_GET["id"];

$sql = "DELETE FROM livros WHERE id=?";
$stmt = $conexao->prepare($sql); //prepara a consulta no banco
$stmt->bind_param("i", $id); // liga a variavel ao ? e i = numero
inteiro
$stmt->execute();

header("Location: ../index.php");
?>

```

Explicação: No código original, o id vem direto da URL (parâmetro GET) e é colocado direto dentro do texto do comando SQL, sem aspas e sem nenhuma verificação. O problema disso é que qualquer pessoa pode editar o final do link manualmente e mudar o que é enviado pro banco, e o banco não tem como saber o que é o comando e o que é o valor digitado. Se alguém colocar outro tipo de conteúdo no lugar do id na URL, dá para alterar o comando inteiro e até apagar outros registros que não deveria. Na versão modificada isso muda porque em vez de colocar o valor direto na string, uso uma interrogação no lugar dele. As funções usadas são as mesmas do cadastro e são aplicadas da mesma forma, só que aqui ligando o id como número inteiro. Isso separa a forma do comando do dado que vai entrar nele. A principal diferença entre as duas formas é que no código original, o valor da URL e o comando ficam misturados no mesmo texto antes de chegar no banco, então o banco não consegue diferenciar um do outro. Já usando parâmetros, o banco recebe primeiro só o comando travado, e só depois recebe o dado separadamente, e esse dado é sempre tratado como informação pura, nunca como parte do comando. É por isso que mesmo que alguém tente editar a URL para colocar outra coisa no lugar do id, o banco só vai procurar por aquele valor exato, e não vai executar ele como se fosse parte de um comando.

- **Atualizar**

Código original:

```

<?php

include "../infra/conexao.php";

$id = $_POST["id"];
$titulo = $_POST["titulo"];
$autor = $_POST["autor"];
$ano = $_POST["ano"];

$sql = "UPDATE livros SET titulo='$titulo',autor='$autor',ano='$ano'
WHERE id = '$id'";

mysqli_query($conexao, $sql);
header("Location: ../index.php");

```

Versão modificada:

```
<?php

include "../infra/conexao.php";

$id = $_POST["id"];
$titulo = $_POST["titulo"];
$autor = $_POST["autor"];
$ano = $_POST["ano"];

$sql = "UPDATE livros SET titulo=?,autor=?,ano=? WHERE id = ?";
$stmt = $conexao->prepare($sql);
$stmt->bind_param("ssii", $titulo, $autor, $ano, $id);
$stmt->execute();

header("Location: ../index.php");
```

Explicação: No código original, os dados do formulário e o id utilizado para identificar o livro a ser atualizado são inseridos diretamente na string do comando SQL entre aspas. Dessa forma, o banco de dados recebe tudo como um único texto e não é capaz de diferenciar o comando dos valores fornecidos pelo usuário. Na versão modificada, todos os valores foram substituídos por ?. Como essa consulta tem quatro parâmetros, a string de tipos usada no bind_param() é "ssii", com o título e autor como texto e o ano e id como números, mantendo a mesma ordem em que aparecem na query. Assim os dados são enviados separadamente do comando, e o banco de dados sempre os trata como valores literais, nunca como parte da instrução SQL.

Conclusão:

A análise do projeto mostrou que um CRUD simples já apresenta os mesmos riscos que podem ser encontrados em sistemas reais quando os dados vindos do usuário são colocados direto dentro dos comandos SQL. As operações analisadas tinham o mesmo problema, nenhuma separação entre o comando e os dados, o que deixava o sistema vulnerável a SQL Injection. Os prepared statements resolvem esse problema porque fazem o banco de dados tratar o texto digitado pelo usuário sempre como dado e nunca como parte do comando. Isso acaba com o risco de alguém alterar a lógica da consulta através de um campo de formulário ou de um parâmetro na URL. Por esse motivo o uso de prepared statements é considerado uma boa prática muito importante em qualquer aplicação que receba dados de usuários e os utilize em comandos para um banco de dados, sendo uma das formas mais simples e eficazes de proteger um sistema contra um dos ataques mais comuns na segurança web.